E4 Educator v2.0

T10

SD card and file I/O

Tier 2 · Tutorial 3 of 9

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How the microSD slot on the TFT module shares the SPI bus
- The critical SPI initialisation order — TFT first, SD second
- How to mount, read, write, and append files on the SD card
- How to log sensor data to a timestamped CSV file
- How to display a BMP splash screen from SD on TFT startup
- How to read a config file from SD to set runtime parameters
- How to manage log files — size limits, rotation, and cleanup
- How to browse the SD card directory listing on the TFT display

You will need:

- E4 Educator v2.0 board with TFT module socketed and ESP32 fitted
- microSD card (2GB-32GB, FAT32 formatted)
- T08 and T09 completed — TFT and touch working
- Button.h, TouchZone.h from previous tutorials

1 The SD card on the SPI bus

The TFT module on the E4 carries a microSD slot on its rear PCB. Like the XPT2046 touch controller, the SD card shares the VSPI bus with the ILI9341 display and uses its own chip select pin to avoid conflicts.

Signal	GPIO / Define	Notes
MOSI	23 / TFT_MOSI	Shared with TFT and touch
MISO	19 / TFT_MISO	Shared — SD card read data returns here
SCLK	18 / TFT_SCLK	Shared SPI clock
SD_CS	15 / SD_CS	SD chip select. 10k pull-up to 3V3 fitted on E4.

1.1 The critical initialisation order

The most common SD card failure in ESP32 projects is incorrect SPI initialisation order. The rule is absolute:

TFT first. SD second. Always.

Call tft.init() before SD.begin(). The TFT_eSPI library configures the SPI bus during tft.init(). If SD.begin() runs first, the SPI bus is not yet correctly configured and the SD card will fail to mount — often silently, with no clear error. This order requirement is not obvious from the library documentation and catches almost every learner the first time.

```
void setup() {
  // Backlight safe init
  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);

  // Step 1: TFT ALWAYS first
  tft.init();
  tft.setRotation(0);   // Landscape — correct for E4 Educator v2.0
  tft.fillScreen(TFT_BLACK);

  // Step 2: Backlight on
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 200);

  // Step 3: SD card AFTER TFT
  if (!SD.begin(SD_CS)) {
    Serial.println("ERROR: SD card mount failed.");
    Serial.println("Check: card inserted? FAT32 formatted? TFT init before SD?");
    // Continue without SD — degrade gracefully
  } else {
    Serial.printf("SD mounted. Size: %lluMB\n",
                  SD.cardSize() / (1024 * 1024));
  }
}
```

1.2 SD card requirements

Requirement	Detail

Format	FAT32. exFAT is not supported by the ESP32 SD library. Cards larger than 32GB may be exFAT by default — reformat to FAT32.
Capacity	2GB to 32GB recommended. 64GB+ cards often require special handling.
Speed class	Class 4 minimum. Class 10 recommended for logging applications.
Voltage	3.3V. The E4 runs at 3V3. Do not use cards requiring 5V signalling.
File names	8.3 format (8 character name + 3 character extension) for widest compatibility. e.g. LOG00001.CSV not sensor_log_2026.csv

2 File operations

The ESP32 SD library provides a familiar File object for reading and writing. The operations are close to standard C++ file I/O with some ESP32-specific behaviour worth understanding.

2.1 Writing and appending

```
#include <Arduino.h>
#include <SPI.h>
#include <SD.h>
#include <TFT_eSPI.h>

TFT_eSPI tft;

// Write a file (creates or overwrites)
bool writeFile(const char* path, const char* content) {
  File f = SD.open(path, FILE_WRITE);
  if (!f) {
    Serial.printf("Failed to open %s for writing\n", path);
    return false;
  }
  f.print(content);
  f.close();
  Serial.printf("Written: %s\n", path);
  return true;
}

// Append to a file (creates if not exists)
bool appendLine(const char* path, const char* line) {
  File f = SD.open(path, FILE_APPEND);
  if (!f) {
    Serial.printf("Failed to open %s for append\n", path);
    return false;
  }
  f.println(line);
  f.close();
  return true;
}

// Read entire file to Serial
void printFile(const char* path) {
  File f = SD.open(path);
  if (!f) {
    Serial.printf("File not found: %s\n", path);
    return;
  }
  Serial.printf("--- %s ---\n", path);
  while (f.available()) {
    Serial.write(f.read());
  }
  f.close();
  Serial.println("--- end ---");
}
```

```
// Delete a file
bool deleteFile(const char* path) {
  if (SD.remove(path)) {
    Serial.printf("Deleted: %s\n", path);
    return true;
  }
  Serial.printf("Delete failed: %s\n", path);
  return false;
}

// Check file exists and get size
void fileInfo(const char* path) {
  File f = SD.open(path);
  if (f) {
    Serial.printf("%s  size: %lu bytes\n", path, f.size());
    f.close();
  } else {
    Serial.printf("%s  not found\n", path);
  }
}
```

Key point

Always call f.close() after every file operation. The SD library buffers writes — if you do not close the file, data may not be flushed to the card. In a logging application this means the last few log entries may be lost on power loss. Close after every append.

2.2 Directory listing

```
void listDir(const char* dirPath) {
  File dir = SD.open(dirPath);
  if (!dir || !dir.isDirectory()) {
    Serial.printf("Not a directory: %s\n", dirPath);
    return;
  }
  Serial.printf("Directory: %s\n", dirPath);
  File entry = dir.openNextFile();
  while (entry) {
    if (entry.isDirectory()) {
      Serial.printf("  [DIR]  %s\n", entry.name());
    } else {
      Serial.printf("  [FILE] %-20s  %lu bytes\n",
                    entry.name(), entry.size());
    }
    entry.close();
    entry = dir.openNextFile();
  }
  dir.close();
}

// Usage: listDir("/");  // Root directory
```

3 CSV sensor data logging

CSV (Comma Separated Values) is the standard format for sensor data logging. Files open directly in Excel, LibreOffice, or any data analysis tool. Each row is one reading with a timestamp and one or more sensor values.

3.1 Log file format

A well-structured log file has a header row on the first line followed by data rows. Use millis() as the timestamp since NTP and RTC are covered in later tutorials — millis() gives elapsed time in milliseconds which is sufficient for relative timing analysis.

```
// CSV format for AHT20 + LDR logging:
millis,tempC,humidity,lightPct
0,21.3,55.2,68
3000,21.4,55.1,67
6000,21.4,55.3,72
...
```

3.2 Complete logging implementation

```
#include <Arduino.h>
#include <Wire.h>
#include <SPI.h>
#include <SD.h>
#include <TFT_eSPI.h>
#include <Adafruit_AHTX0.h>
#include "Button.h"

TFT_eSPI       tft;
TFT_eSprite    spr = TFT_eSprite(&tft);
Adafruit_AHTX0 aht;
Button         nextBtn(BTN_NEXT);
Button         entBtn(BTN_ENT);

#define COL_BG     0x0000
#define COL_HEADER 0x0319
#define COL_TEXT   0xFFFF
#define COL_OK     0x07E0
#define COL_WARN   0xFFE0
#define COL_ERR    0xF800
#define COL_LABEL  0xC618

#define LOG_FILE   "/LOG00001.CSV"
#define MAX_LOG_KB 512            // Rotate log when > 512KB

bool    sdOk     = false;
bool    logging  = true;
long    logCount = 0;
float   tempC    = 0.0, humidity = 0.0;
int     lightPct = 0;
```

```
unsigned long lastLog    = 0;
unsigned long lastDisplay = 0;
const unsigned long LOG_INTERVAL     = 5000;
const unsigned long DISPLAY_INTERVAL = 1000;

// ── LDR ──────────────────────────────────────────────────────
int readLDR() {
  long t = 0;
  for (int i = 0; i < 8; i++) { t += analogRead(LDR); delay(2); }
  return constrain(map(t/8, 200, 3800, 0, 100), 0, 100);
}

// ── Log file management ──────────────────────────────────────
void initLogFile() {
  if (!sdOk) return;
  if (!SD.exists(LOG_FILE)) {
    // Create with header row
    File f = SD.open(LOG_FILE, FILE_WRITE);
    if (f) {
      f.println("millis,tempC,humidity,lightPct");
      f.close();
      Serial.printf("Log created: %s\n", LOG_FILE);
    }
  } else {
    // Count existing rows
    File f = SD.open(LOG_FILE);
    if (f) {
      while (f.available()) {
        if (f.read() == '\n') logCount++;
      }
      f.close();
      logCount--;  // Subtract header row
      Serial.printf("Existing log: %ld rows\n", logCount);
    }
  }
}

bool checkLogSize() {
  if (!sdOk) return false;
  File f = SD.open(LOG_FILE);
  if (!f) return false;
  unsigned long sz = f.size();
  f.close();
  return (sz > (unsigned long)MAX_LOG_KB * 1024);
}

void rotateLog() {
  // Rename existing log and start fresh
  char bakName[20];
  snprintf(bakName, sizeof(bakName), "/LOG_BAK.CSV");
  if (SD.exists(bakName)) SD.remove(bakName);
  SD.rename(LOG_FILE, bakName);
  logCount = 0;
  initLogFile();
```

```
  Serial.println("Log rotated.");
}

bool logReading() {
  if (!sdOk || !logging) return false;
  if (checkLogSize()) rotateLog();

  char line[64];
  snprintf(line, sizeof(line), "%lu,%.2f,%.1f,%d",
           millis(), tempC, humidity, lightPct);

  File f = SD.open(LOG_FILE, FILE_APPEND);
  if (!f) return false;
  f.println(line);
  f.close();
  logCount++;
  return true;
}

// ── Display ──────────────────────────────────────────────
void drawChrome() {
  tft.fillScreen(COL_BG);
  tft.fillRect(0, 0, 240, 40, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("SD Data Logger");
}

void updateDisplay() {
  spr.fillSprite(COL_BG);

  // Sensor values
  spr.setTextColor(COL_TEXT, COL_BG);
  spr.setTextSize(1);
  spr.setCursor(0, 0);
  spr.print("Temperature:");
  spr.setTextSize(3);
  spr.setCursor(0, 14);
  spr.printf("%.1f C", tempC);

  spr.setTextSize(1);
  spr.setCursor(0, 60);
  spr.print("Humidity:");
  spr.setTextSize(3);
  spr.setCursor(0, 74);
  spr.printf("%.1f %%", humidity);

  spr.setTextSize(1);
  spr.setCursor(0, 120);
  spr.printf("Light: %d%%", lightPct);

  // SD status
  spr.setCursor(0, 145);
```

```
  spr.setTextColor(sdOk ? COL_OK : COL_ERR, COL_BG);
  spr.printf("SD: %s", sdOk ? "OK" : "FAIL");

  // Log status
  spr.setCursor(0, 162);
  spr.setTextColor(logging ? COL_OK : COL_WARN, COL_BG);
  spr.printf("Log: %s  Rows: %ld",
             logging ? "ON " : "OFF", logCount);

  // Button hints
  spr.setTextColor(COL_LABEL, COL_BG);
  spr.setCursor(0, 200);
  spr.print("NEXT=force read");
  spr.setCursor(0, 215);
  spr.print("ENT short=log on/off");
  spr.setCursor(0, 230);
  spr.print("ENT long=delete log");

  spr.pushSprite(0, 48);
}

// ── Setup ──────────────────────────────────────────────────
void setup() {
  Serial.begin(115200);
  Serial.printf("T10 SD Logger  FW:%s\n", FW_VERSION);

  Wire.begin(I2C_SDA, I2C_SCL);

  pinMode(LED_GREEN, OUTPUT); digitalWrite(LED_GREEN, LOW);
  pinMode(LED_RED,   OUTPUT); digitalWrite(LED_RED,   LOW);
  pinMode(LED_BLUE,  OUTPUT); digitalWrite(LED_BLUE,  LOW);

  nextBtn.begin();
  entBtn.begin();

  // TFT FIRST
  pinMode(TFT_BL, OUTPUT); digitalWrite(TFT_BL, LOW);
  tft.init();
  tft.setRotation(0);
  tft.fillScreen(COL_BG);
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, 200);

  spr.setColorDepth(16);
  spr.createSprite(320, 165);

  // SD SECOND
  sdOk = SD.begin(SD_CS);
  if (!sdOk) {
    Serial.println("SD mount failed — logging disabled.");
    digitalWrite(LED_RED, HIGH);
  } else {
```

```
    Serial.printf("SD: %.0fMB free of %.0fMB\n",
      (float)SD.totalBytes() / 1048576,
      (float)SD.usedBytes()  / 1048576);
    initLogFile();
    digitalWrite(LED_GREEN, HIGH);
  }

  if (!aht.begin(&Wire)) {
    Serial.println("AHT20 not found.");
  }

  drawChrome();
  updateDisplay();
  Serial.println("Ready.");
}

// ── Loop ──────────────────────────────────────────────────────
void loop() {
  unsigned long now = millis();
  Button::Event nextEv = nextBtn.read();
  Button::Event entEv  = entBtn.read();

  // NEXT: force immediate read and log
  bool shouldLog = (nextEv == Button::SHORT_PRESS);

  // ENT short: toggle logging on/off
  if (entEv == Button::SHORT_PRESS) {
    logging = !logging;
    digitalWrite(LED_BLUE, logging ? HIGH : LOW);
    Serial.printf("Logging: %s\n", logging ? "ON" : "OFF");
    updateDisplay();
  }

  // ENT long: delete log file
  if (entEv == Button::LONG_PRESS && sdOk) {
    SD.remove(LOG_FILE);
    logCount = 0;
    initLogFile();
    Serial.println("Log deleted and reset.");
    updateDisplay();
  }

  // Timed read and log
  if (now - lastLog >= LOG_INTERVAL) shouldLog = true;

  if (shouldLog) {
    lastLog = now;
    sensors_event_t h, t;
    aht.getEvent(&h, &t);
    tempC    = t.temperature;
    humidity = h.relative_humidity;
    lightPct = readLDR();

    if (logReading()) {
```

```
      Serial.printf("[%ld] Logged: %.1fC  %.1f%%  %d%%\n",
                    logCount, tempC, humidity, lightPct);
    }
    updateDisplay();
  }

  // Update display every second for uptime
  if (now - lastDisplay >= DISPLAY_INTERVAL) {
    lastDisplay = now;
    updateDisplay();
  }
}
```

4 BMP splash screen from SD

Loading a bitmap image from SD and displaying it on the TFT as a startup splash screen is one of the most satisfying early milestones in embedded display programming. It looks professional, teaches file I/O, and demonstrates the SPI bus sharing in a compelling way.

4.1 Preparing the BMP file

The BMP must be in 24-bit uncompressed format and sized to fit the landscape display (240×320 for portrait). Use any image editor:

- In GIMP: Image → Scale Image → 240×320. Export As → .bmp → 24-bit uncompressed.
- In Paint.NET: Image → Resize → 240×320. File → Save As → BMP → 24-bit.
- In Photoshop: Image → Image Size → 240×320. Save As → BMP → 24-bit.

Save as SPLASH.BMP in the root of the SD card.

4.2 BMP loading function

```
// Draw a 24-bit BMP from SD to TFT at position (x, y)
// Handles BMP header parsing and row-by-row pixel output
bool drawBMP(const char* filename, int x, int y) {
  File bmpFile = SD.open(filename);
  if (!bmpFile) {
    Serial.printf("BMP not found: %s\n", filename);
    return false;
  }

  // Read BMP header
  if (bmpFile.read() != 'B' || bmpFile.read() != 'M') {
    Serial.println("Not a BMP file.");
    bmpFile.close();
    return false;
  }

  // Skip file size (4 bytes) and reserved (4 bytes)
  bmpFile.seek(10);
  uint32_t dataOffset = 0;
  bmpFile.read((uint8_t*)&dataOffset, 4);

  // Read image dimensions from DIB header
  bmpFile.seek(18);
  int32_t bmpW = 0, bmpH = 0;
  bmpFile.read((uint8_t*)&bmpW, 4);
  bmpFile.read((uint8_t*)&bmpH, 4);

  // Check bit depth (must be 24-bit)
  bmpFile.seek(28);
  uint16_t bpp = 0;
  bmpFile.read((uint8_t*)&bpp, 2);
  if (bpp != 24) {
    Serial.printf("BMP must be 24-bit. Found: %d-bit\n", bpp);
```

```
    bmpFile.close();
    return false;
  }

  Serial.printf("BMP: %dx%d px, 24-bit\n", bmpW, bmpH);

  // BMP rows are bottom-up and padded to 4-byte boundaries
  int rowSize = ((bmpW * 3) + 3) & ~3;
  uint8_t rowBuf[rowSize];

  tft.startWrite();
  for (int row = bmpH - 1; row >= 0; row--) {
    bmpFile.seek(dataOffset + row * rowSize);
    bmpFile.read(rowBuf, bmpW * 3);

    // Convert BGR (BMP) to RGB565 and push each pixel
    for (int col = 0; col < bmpW; col++) {
      uint8_t b = rowBuf[col * 3];
      uint8_t g = rowBuf[col * 3 + 1];
      uint8_t r = rowBuf[col * 3 + 2];
      uint16_t colour = tft.color565(r, g, b);
      tft.drawPixel(x + col, y + row, colour);
    }
  }
  tft.endWrite();
  bmpFile.close();
  Serial.println("BMP drawn.");
  return true;
}

// In setup() — after TFT init and SD mount:
  if (sdOk) {
    drawBMP("/SPLASH.BMP", 0, 0);
    delay(2000);  // Show splash for 2 seconds
    tft.fillScreen(TFT_BLACK);  // Clear before main UI
  }
```

Key point

BMP files store pixels in BGR order (Blue, Green, Red) from bottom to top. The conversion to RGB565 swaps B and R and the row loop reads from bmpH-1 down to 0 to correct the vertical flip. Both of these are standard BMP format characteristics — not bugs.

5 Reading a config file from SD

Storing configuration in a text file on the SD card rather than hardcoded in firmware gives you a device that can be reconfigured without reflashing. This is how industrial data loggers and field instruments work.

5.1 Config file format

A simple key=value format is easy to parse and easy to edit on a PC:

```
# E4 Logger configuration
# Lines starting with # are comments
log_interval=10000
temp_alert=28.0
humid_alert=70.0
log_enabled=1
brightness=200
```

5.2 Config parser

```
struct Config {
  unsigned long logInterval  = 5000;
  float         tempAlert    = 28.0;
  float         humidAlert   = 70.0;
  bool          logEnabled   = true;
  int           brightness   = 200;
};

Config cfg;

void loadConfig() {
  File f = SD.open("/CONFIG.TXT");
  if (!f) {
    Serial.println("CONFIG.TXT not found — using defaults.");
    return;
  }

  while (f.available()) {
    String line = f.readStringUntil('\n');
    line.trim();

    // Skip comments and empty lines
    if (line.startsWith("#") || line.length() == 0) continue;

    int eqPos = line.indexOf('=');
    if (eqPos < 0) continue;

    String key   = line.substring(0, eqPos);
    String value = line.substring(eqPos + 1);
    key.trim();
    value.trim();

    if (key == "log_interval") cfg.logInterval = value.toInt();
```

```
    else if (key == "temp_alert")  cfg.tempAlert   = value.toFloat();
    else if (key == "humid_alert") cfg.humidAlert  = value.toFloat();
    else if (key == "log_enabled") cfg.logEnabled  = (value == "1");
    else if (key == "brightness")  cfg.brightness  = value.toInt();
  }
  f.close();

  Serial.printf("Config loaded: interval=%lums temp>%.1f humid>%.1f\n",
                cfg.logInterval, cfg.tempAlert, cfg.humidAlert);
}

// In setup(), call loadConfig() after SD.begin():
  if (sdOk) {
    loadConfig();
    logging = cfg.logEnabled;
    ledcWrite(0, cfg.brightness);
  }
```

Graceful degradation

Always design SD-dependent features to degrade gracefully if the SD card is absent or fails. The config loader above falls through to defaults if CONFIG.TXT is not found. The logger disables itself if SD.begin() fails. A device that refuses to work without an SD card is fragile — a device that continues with reduced functionality is robust.

6 Log file management

A data logger that runs continuously will eventually fill the SD card or produce files too large to open conveniently. Good log management handles this automatically.

6.1 Size-based rotation strategy

Strategy	When to use
Size limit rotation	Rename current log to BAK when it exceeds a size limit, start fresh. Simple, keeps two files. Already implemented in Section 3.2.
Numbered rotation	LOG00001.CSV, LOG00002.CSV etc. Keep N files, delete oldest when limit reached. Better for long-term deployments.
Daily rotation	New file per day using RTC date. Requires DS3231 RTC (covered in Tier 3). Best for timestamped environmental monitoring.
Fixed circular buffer	Write to fixed-size file, overwrite oldest rows. Complex but zero card fill risk. Rarely needed for educational projects.

6.2 Estimating log file growth

Plan your log size so you know how long a card will last before rotation:

Log interval	Row size	Rows/day	MB/day
1 second	`~40 bytes`	86,400	~3.3 MB
5 seconds	`~40 bytes`	17,280	~0.66 MB
1 minute	`~40 bytes`	1,440	~0.055 MB
5 minutes	`~40 bytes`	288	~0.011 MB

A 5-second logging interval on a 4GB card would take approximately 16 years to fill — the 512KB rotation limit in the Section 3.2 implementation means rotation happens roughly every 800 readings, keeping files manageable for analysis.

7 T10 summary

Concept	Key takeaway
SPI init order	TFT first. SD second. Always. Without this order, SD.begin() fails because TFT_eSPI has not yet configured the SPI bus.
SD_CS pull-up	GPIO 15 has 10k pull-up to 3V3 on the E4. This keeps SD_CS HIGH at boot, preventing boot mode interference.
FAT32 format	SD card must be FAT32. Cards over 32GB may default to exFAT — reformat to FAT32 before use.
f.close()	Always close files after every operation. Unclosed files lose buffered writes on power loss.
CSV logging	Header row on first line. One row per reading. millis() as timestamp. snprintf() builds lines without String heap fragmentation.
BMP splash	24-bit uncompressed BMP only. BGR pixel order, rows bottom-up. tft.color565() converts each pixel. Parse header to find data offset.

Config files	key=value text file. Parse with indexOf('='). Fall through to defaults if file absent. Always design for graceful degradation.
Log rotation	Check file size before each append. Rename to BAK and start fresh when limit exceeded. Plan: 5s interval ≈ 0.66MB/day.

What's next

- T11 — NVS persistence: now that SD card config is established, NVS (Non-Volatile Storage) provides a lighter-weight alternative for small settings that need to survive reboots without a card present. Touch calibration in T09 already used NVS — T11 explores it fully.
- T12 — WiFi and MQTT: with logging established locally to SD, the natural next step is sending data to a network broker. The T08 dashboard becomes a networked sensor node.